Dan Pham

CSE536

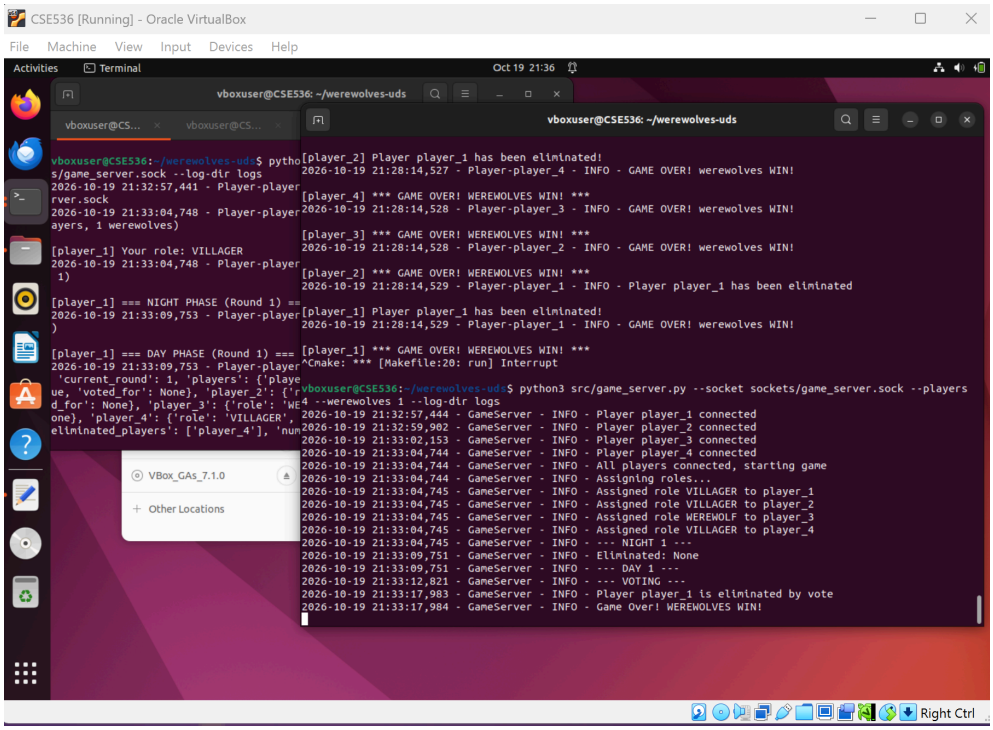
10/19/26

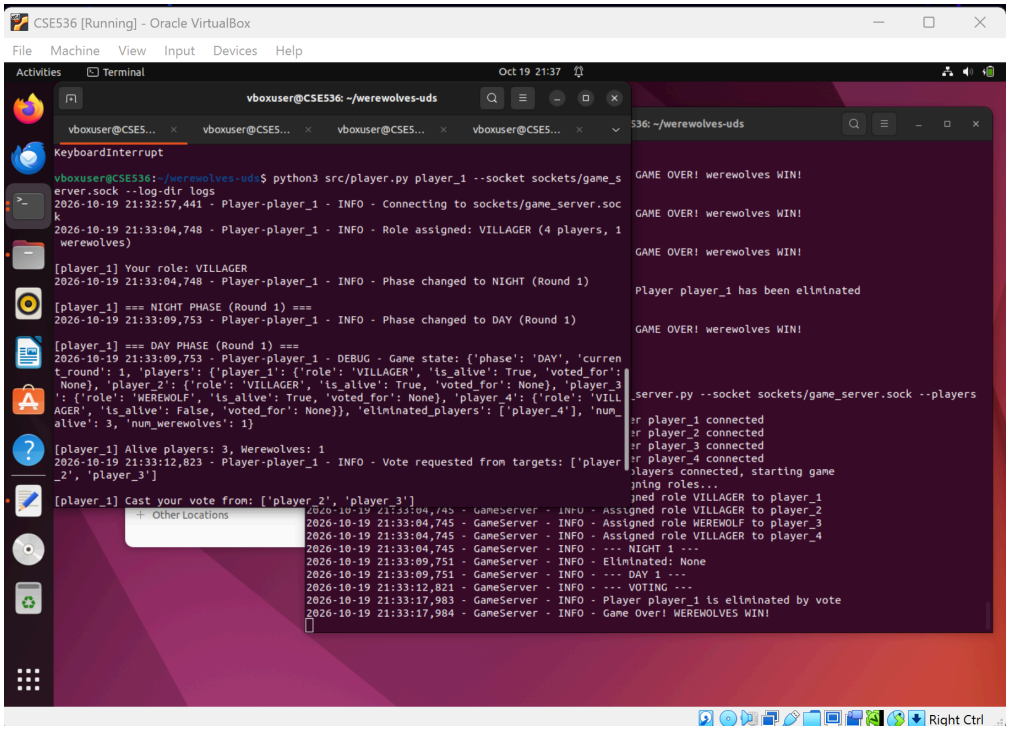
Project 1: Werewolves-like Game

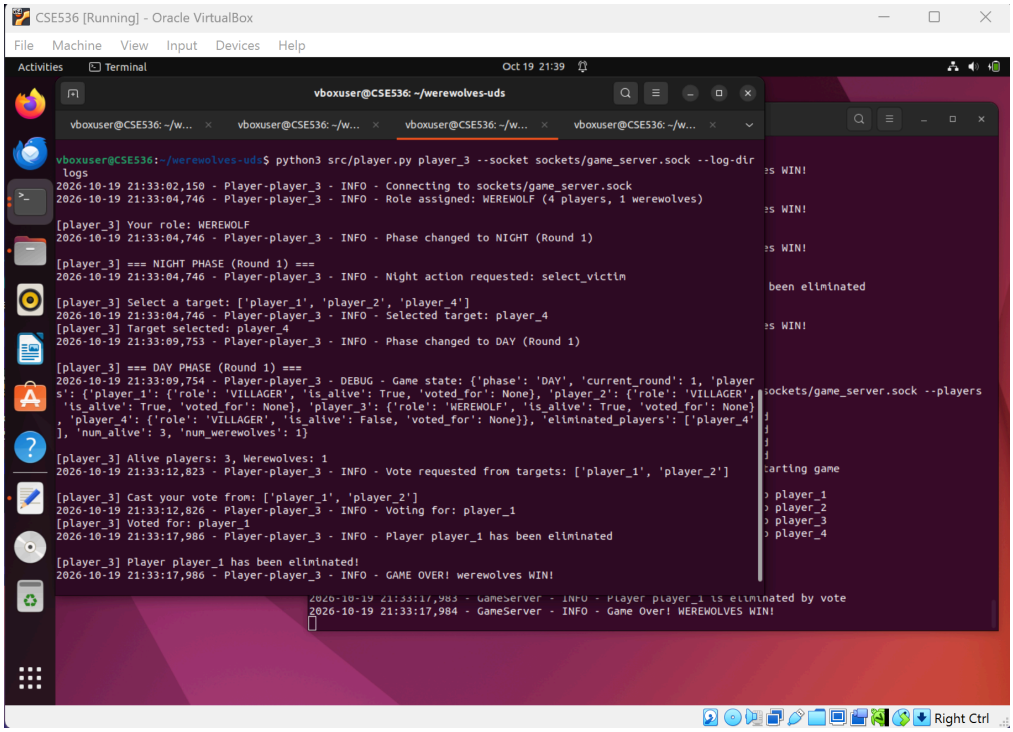
- Introduction:

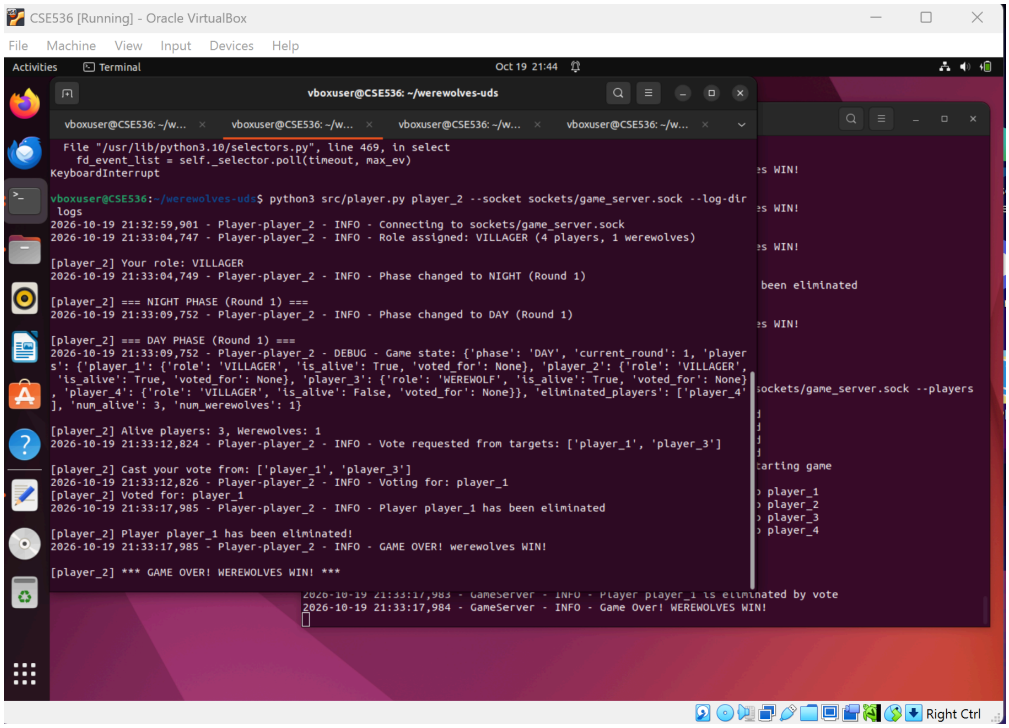
The Werewolves game is a social deduction game where players take on hidden roles (werewolves vs villagers) and compete through discussion and voting. This implementation uses Unix Domain Sockets (UDS) for inter-process communication, with a central game server coordinating game flow and state synchronization across multiple player processes. The communications library is contained in a single file: comms.py, which provides UDSServer and UDSClient classes for async message passing with broadcast support.

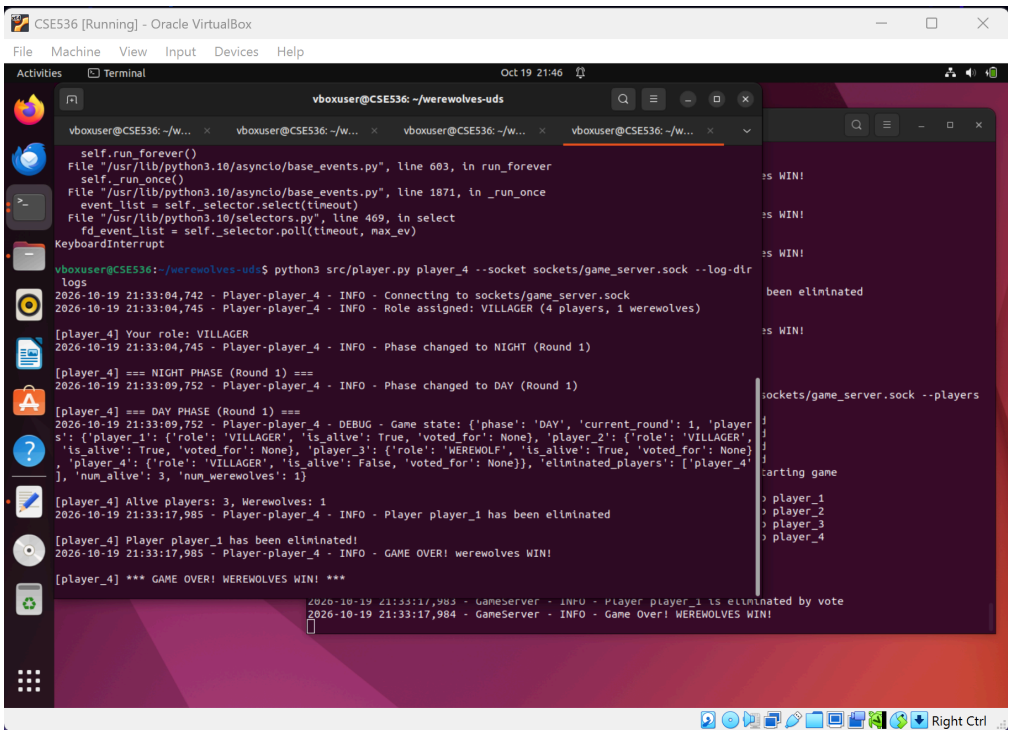
- In-Game screenshots:




CSE536 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Oct 19 21:37
vboxuser@CSE536: ~/werewolves-uds
KeyboardInterrupt
vboxuser@CSE536:~/werewolves-uds$ python3 src/player.py player_1 --socket sockets/game_server.sock --log-dir logs
2026-10-19 21:32:57,441 - Player-player_1 - INFO - Connecting to sockets/game_server.sock
2026-10-19 21:33:04,748 - Player-player_1 - INFO - Role assigned: VILLAGER (4 players, 1 werewolves)
[player_1] Your role: VILLAGER
2026-10-19 21:33:04,748 - Player-player_1 - INFO - Phase changed to NIGHT (Round 1)
[player_1] === NIGHT PHASE (Round 1) ===
2026-10-19 21:33:09,753 - Player-player_1 - INFO - Phase changed to DAY (Round 1)
[player_1] === DAY PHASE (Round 1) ===
2026-10-19 21:33:09,753 - Player-player_1 - DEBUG - Game state: {'phase': 'DAY', 'current_round': 1, 'players': {'player_1': {'role': 'VILLAGER', 'is_alive': True, 'voted_for': None}, 'player_2': {'role': 'VILLAGER', 'is_alive': True, 'voted_for': None}, 'player_3': {'role': 'WEREWOLF', 'is_alive': True, 'voted_for': None}, 'player_4': {'role': 'VILLAGER', 'is_alive': False, 'voted_for': None}}, 'eliminated_players': ['player_4'], 'num_alive': 3, 'num_werewolves': 1}
[player_1] Alive players: 3, Werewolves: 1
2026-10-19 21:33:12,823 - Player-player_1 - INFO - Vote requested from targets: ['player_2', 'player_3']
[player_1] Cast your vote from: ['player_2', 'player_3']
GAME OVER! werewolves WIN!
Player player_1 has been eliminated
server.py --socket sockets/game_server.sock --players
player_1 connected
player_2 connected
player_3 connected
player_4 connected
players connected, starting game
Other Locations
2026-10-19 21:33:04,745 - GameServer - INFO - Assigned role VILLAGER to player_1
2026-10-19 21:33:04,745 - GameServer - INFO - Assigned role VILLAGER to player_2
2026-10-19 21:33:04,745 - GameServer - INFO - Assigned role WEREWOLF to player_3
2026-10-19 21:33:04,745 - GameServer - INFO - Assigned role VILLAGER to player_4
2026-10-19 21:33:04,745 - GameServer - INFO - --- NIGHT 1 ---
2026-10-19 21:33:09,751 - GameServer - INFO - Eliminated: None
2026-10-19 21:33:09,751 - GameServer - INFO - --- DAY 1 ---
2026-10-19 21:33:12,821 - GameServer - INFO - --- VOTING ---
2026-10-19 21:33:17,983 - GameServer - INFO - Player player_1 is eliminated by vote
2026-10-19 21:33:17,984 - GameServer - INFO - Game Over! WEREWOLVES WIN!
Right Ctrl


CSE536 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Oct 19 21:39
vboxuser@CSE536: ~/werewolves-uds
vboxuser@CSE536:~/werewolves-uds$ python3 src/player.py player_3 --socket sockets/game_server.sock --log-dir logs
2026-10-19 21:33:02,150 - Player-player_3 - INFO - Connecting to sockets/game_server.sock
2026-10-19 21:33:04,746 - Player-player_3 - INFO - Role assigned: WEREWOLF (4 players, 1 werewolves)
[player_3] Your role: WEREWOLF
2026-10-19 21:33:04,746 - Player-player_3 - INFO - Phase changed to NIGHT (Round 1)
[player_3] === NIGHT PHASE (Round 1) ===
2026-10-19 21:33:04,746 - Player-player_3 - INFO - Night action requested: select_victim
[player_3] Select a target: ['player_1', 'player_2', 'player_4']
2026-10-19 21:33:04,746 - Player-player_3 - INFO - Selected target: player_4
[player_3] Target selected: player_4
2026-10-19 21:33:09,753 - Player-player_3 - INFO - Phase changed to DAY (Round 1)
[player_3] === DAY PHASE (Round 1) ===
2026-10-19 21:33:09,754 - Player-player_3 - DEBUG - Game state: {'phase': 'DAY', 'current_round': 1, 'players': {'player_1': {'role': 'VILLAGER', 'is_alive': True, 'voted_for': None}, 'player_2': {'role': 'VILLAGER', 'is_alive': True, 'voted_for': None}, 'player_3': {'role': 'WEREWOLF', 'is_alive': True, 'voted_for': None}, 'player_4': {'role': 'VILLAGER', 'is_alive': False, 'voted_for': None}}, 'eliminated_players': ['player_4'], 'num_alive': 3, 'num_werewolves': 1}
[player_3] Alive players: 3, Werewolves: 1
2026-10-19 21:33:12,823 - Player-player_3 - INFO - Vote requested from targets: ['player_1', 'player_2']
[player_3] Cast your vote from: ['player_1', 'player_2']
2026-10-19 21:33:12,826 - Player-player_3 - INFO - Voting for: player_1
[player_3] Voted for: player_1
2026-10-19 21:33:17,986 - Player-player_3 - INFO - Player player_1 has been eliminated
[player_3] Player player_1 has been eliminated!
2026-10-19 21:33:17,986 - Player-player_3 - INFO - GAME OVER! werewolves WIN!
been eliminated
sockets/game_server.sock --players
player_1
player_2
player_3
player_4
2026-10-19 21:33:17,984 - GameServer - INFO - Game Over! WEREWOLVES WIN!
Right Ctrl





- Chosen Primitive & How it works:

Unix Domain Sockets (UDS) are a more sophisticated IPC primitive than named pipes because they support full-duplex (bidirectional) communication on a single socket, connection-oriented semantics similar to TCP, and efficient async I/O integration with asyncio. The implementation uses a length-prefixed message protocol (4-byte length header + JSON payload) to ensure proper message framing. The server handles multiple concurrent client connections via asyncio tasks, where each client task independently reads messages, processes them, and triggers registered handlers. Broadcast is implemented by iterating over all connected clients and sending the same message. This provides sophisticated communication patterns including unicast (1-to-1), multicast (1-to-specific group), and broadcast (1-to-all) - all on top of a single UDS.

- Closing:

This project demonstrated that Unix Domain Sockets provide an excellent balance between simplicity and sophistication for local IPC. The async/await integration with Python's asyncio made concurrent message handling elegant and efficient. A key learning was that sophisticated communication patterns (broadcast, multicast) aren't intrinsic to the socket primitive itself - they require application-level logic. For the Werewolves game, I needed to explicitly implement broadcast by maintaining a client registry and sending to all. This helped me understand that the "sophistication" of a primitive isn't just about its raw capabilities, but how well it composes with higher-level patterns. While the game didn't strictly require multicast (all broadcasts went to all players), supporting it taught me about filtering and selective message routing.